HEALTHCLOUD COMPACTO
Plataforma Distribuída de Serviços de Saúde

Roteiro de Proposta e Especificação de Arquitetura Evolutiva

Contexto Acadêmico:	Projeto Integrado para as disciplinas TES (Graduação) e TSD (Pós-Graduação)
Instituição:	Universidade Federal do Piauí (UFPI) - Campus Senador Helvídio Nunes de Barros
Docente Responsável:	Profª Dra. Juliana Oliveira de Carvalho
Período Letivo:	2026.2
Discente Proponente:	Mestrando & Graduando em Engenharia de Software / Sistemas Distribuídos

1. Visão Geral do Projeto e Justificativa de Escopo

Este documento apresenta a proposta arquitetural para o desenvolvimento do projeto **HealthCloud**, concebido como um esforço prático integrador para atender concomitantemente aos requisitos avaliativos das disciplinas de Tópicos Especiais em Engenharia de Software (TES - nível graduação) e Tópicos Avançados em Sistemas Distribuídos (TSD - nível pós-graduação), ambas sob a tutela da Proponente e Docente Dra. Juliana Oliveira de Carvalho no período de 2026.2. A premissa central do HealthCloud é simular uma plataforma distribuída de telessaúde focada no acompanhamento clínico remoto e na análise preditiva elementar de indicadores de saúde baseada em dados sintéticos.

A especificação original do projeto prevê uma decomposição em até 7 microsserviços (API Gateway, Auth, User, Patient, Health Record, Analytics, Notification). No entanto, para viabilizar o desenvolvimento completo, testes experimentais e implantação por um único discente, esta proposta apresenta uma **redefinição de escopo compacta com apenas 3 microsserviços principais**. Esta simplificação visa reduzir o overhead de programação e focar a complexidade de forma cirúrgica na infraestrutura, na comunicação distribuída, na latência de rede inter-cloud e na observabilidade do sistema, garantindo um alinhamento pedagógico rigoroso com os objetivos das disciplinas.

2. Objetivos e Alinhamento Pedagógico

O objetivo central deste roteiro é projetar e documentar a evolução de uma arquitetura modular de baixo acoplamento para o HealthCloud, minimizando o retrabalho operacional através de um ecossistema unificado. Os objetivos específicos são organizados por disciplina:

- **Objetivos em TES (Graduação):** Projetar e construir uma aplicação baseada em microsserviços utilizando boas práticas de isolamento (banco de dados por serviço); containerizar os componentes utilizando Docker para garantir portabilidade; e implantar a aplicação na nuvem principal (AWS) nas modalidades sem contêiner (EC2 puro) e com contêiner (Docker), avaliando o overhead de desempenho induzido pela virtualização de SO.
- **Objetivos em TSD (Pós-Graduação):** Avaliar a latência de comunicação de rede inter-cloud distribuindo geograficamente e entre provedores distintos (AWS e Google Cloud Platform) os microsserviços; estabelecer uma via de comunicação assíncrona baseada em eventos/mensageria; e realizar testes de estresse com cargas concorrentes simultâneas para capturar gargalos de escalabilidade vertical e horizontal.

3. Especificação da Arquitetura Compacta (3 Microsserviços)

A fim de evitar o antipadrão do 'monólito distribuído' (onde microsserviços são criados por atores, como paciente ou médico, ficando altamente acoplados), a divisão proposta é baseada estritamente no princípio de **Capacidade de Negócio e Subdomínios (Bounded Contexts)**. Cada um dos 3 microsserviços abaixo é independente, auto-contido, possui seu próprio ciclo de vida e seu próprio mecanismo de persistência.

1. Portal & Auth Service (Serviço de Interface e Identidade):
Este serviço atua como o ponto de entrada da aplicação para os usuários (médicos e pacientes). É responsável por servir o frontend dinâmico em React/Next.js, intermediar as requisições para as APIs de backend e prover o fluxo de controle de identidade. Implementa a geração de tokens de segurança baseados no padrão JWT (JSON Web Token), armazenando de forma segura credenciais e perfis de usuário em uma tabela de banco de dados SQL isolada. Hospedado primariamente na nuvem AWS.

2. Clinical Core Service (Serviço Clínico Transacional):
Este é o núcleo transacional de saúde da aplicação. Gerencia o cadastro de pacientes (utilizando dados sintéticos estruturados) e registra o histórico de acompanhamento (pressão arterial, glicemia, peso, data e observações clínicas). Possui banco de dados relacional PostgreSQL exclusivo, acessado via ORM Prisma para garantir integridade e agilidade transacional. Hospedado na AWS, expõe endpoints HTTP/REST rápidos para o Portal.

3. Analytics & Alert Service (Serviço de Análise e Monitoramento):
Serviço responsável por computar métricas de saúde agregadas e identificar possíveis anomalias nos sinais vitais de pacientes. Sempre que um novo registro clínico é inserido no sistema, este serviço

analisa se os valores extrapolam limites seguros e dispara alertas no painel. Adicionalmente, consolida dados estatísticos para o dashboard. Hospedado no Google Cloud Platform (GCP) para viabilizar o cenário prático multinuvem de TSD.

Microsserviço	Tecnologia Principal	Provedor Principal	Papel nos Experimentos
Portal & Auth	Next.js / TypeScript / JWT	AWS (EC2 / ECS)	Ponto de contato e baseline local
Clinical Core	Node.js / Express / PostgreSQL	AWS (EC2 / ECS / RDS)	Transação clínica e baseline de nuvem
Analytics & Alert	Node.js (ou Python) / SQLite (Local)	GCP (Compute Engine)	Gargalo de latência inter-cloud

4. Integração de Dados e Estratégia de Comunicação inter-cloud

Um dos pilares fundamentais da arquitetura de microsserviços é a independência de persistência. Neste projeto, **não haverá compartilhamento de banco de dados**. O Portal & Auth Service possui suas tabelas de controle de acesso isoladas; o Clinical Core Service detém os registros transacionais de prontuários; e o Analytics armazena os limites e histórico de alertas disparados em seu banco local rápido SQLite.

Para integrar os microsserviços de forma coordenada e obter as métricas experimentais exigidas pela disciplina de TSD, o fluxo de comunicação será híbrido:

- **Comunicação Síncrona (Portal —> Clinical Core):** Realizada via requisições HTTP/REST tradicionais. Como o Portal precisa listar instantaneamente os pacientes e dados clínicos em tela, as chamadas de leitura (GET) e escrita (POST) de registros clínicos ocorrem em canal síncrono.
- **Comunicação Assíncrona baseada em Eventos (Clinical Core —> Analytics):** A fim de não impactar o tempo de resposta percebido pelo usuário ao salvar um prontuário na AWS, a análise clínica e geração de alertas serão totalmente desacopladas. Sempre que um registro de saúde é criado com sucesso na AWS, o Clinical Core dispara um evento assíncrono. Na modalidade local/mononuvem, isso pode ser integrado via chamada HTTP paralela assíncrona (*non-blocking fire-and-forget*). Na modalidade multinuvem avançada de TSD, utilizaremos um Message Broker leve (como um container RabbitMQ ou o serviço serverless Cloud Pub/Sub do GCP) para garantir a entrega confiável da mensagem cruzando os dois provedores de nuvem (AWS GCP).

5. Roteiro Evolutivo de Implantação e Testes

Para garantir o cumprimento rigoroso dos cronogramas de avaliações, o desenvolvimento e os experimentos práticos do HealthCloud serão conduzidos de forma estritamente evolutiva, subdivididos em duas fases principais:

5.1 Fase Mononuvem: Foco na Graduação (TES)

Focada nos Trabalhos da Terceira Avaliação de TES, esta fase busca demonstrar a proficiência em virtualização, conteinerização e infraestrutura na AWS. O aluno implementará e avaliará:

- **Implantação Pura (Trabalho 3.1):** Instanciação dos microsserviços em servidores virtuais Amazon EC2 independentes e limpos, configurando o ambiente de execução manual do Node.js/Next.js.
- **Implantação em Contêineres (Trabalho 3.2):** Empacotamento de cada um dos 3 microsserviços em imagens Docker otimizadas. Utilização do Docker Compose para orquestrar a infraestrutura na nuvem de forma declarativa.
- **Análise Comparativa (Trabalho 3.3):** Escrita de artigo científico comparando o consumo de recursos computacionais (memória e CPU) e o tempo de resposta médio da aplicação entre a implantação limpa versus a virtualização por contêineres na AWS.

5.2 Fase Multinuvem e Testes de Concorrência: Foco na Pós-Graduação (TSD)

Para a avaliação final de TSD, a infraestrutura é estendida para explorar limites de distribuição e escala. Os testes serão orientados a responder como a barreira física de rede entre provedores afeta o desempenho distribuído:

- **Cenário Multinuvem Distribuído:** Manutenção do Portal e do Core na AWS, enquanto o serviço Analytics & Alert é migrado para o Google Cloud Platform (GCP). A comunicação inter-cloud testará o overhead de latência de tráfego de dados na WAN pública.
- **Simulação de Elasticidade e Carga (k6):** Utilização da ferramenta de testes de carga k6 para injetar acessos simultâneos concorrentes de usuários, elevando progressivamente a carga de 10 para 500 requisições por segundo. Serão aferidas métricas de latência média, percentis críticos (p50, p95, p99) e taxas de erro HTTP para fundamentar os resultados de desempenho.
- **Artigo de Mestrado PPGCC:** Redação do artigo final de TSD (com entrega e apresentação marcadas para 11/12), sintetizando toda a metodologia experimental, análise de custos sob o modelo de provedores integrados, e mapeamento das lacunas de rede inter-cloud observadas.

Esta redefinição para um escopo compacto de 3 microsserviços garante que 100% das competências em computação em nuvem, virtualização e sistemas distribuídos exigidas pela Profª Juliana Carvalho sejam plenamente consolidadas, reduzindo de forma drástica os riscos de complexidade de software desnecessária e permitindo que o esforço acadêmico se concentre na excelência da avaliação de desempenho em nuvem.